CONTENTS

LIST OF TABLES

LIST OF FIGURES

Chapter 1

INTRODUCTION

The Connected Vehicle Data Analysis project explores the potential of Internet of Things (IoT) technology in transforming the automotive industry by leveraging real-time data generated by vehicles. As vehicles become increasingly connected, they generate vast amounts of data that cover parameters such as speed, location, fuel efficiency, engine performance, and driving patterns. These IoT-enabled systems allow continuous data flow from vehicle sensors to central processing units, turning raw data into meaningful insights.

The main focus of this project is to utilize the power of machine learning algorithms to analyze the collected vehicle data. By applying algorithms like Random Forest, Support Vector Machines (SVM), and Gradient Boosting, the system can identify patterns that traditional analytics tools may miss. These techniques can uncover hidden insights regarding vehicle performance, driver behavior, and environmental influences, ultimately enhancing safety and optimizing fuel consumption.

Moreover, by analyzing connected vehicle data, the project aims to predict vehicle maintenance needs before they become critical, reducing breakdowns and costly repairs. The insights generated can also improve traffic flow by predicting congestion or accidents in real time. Integrating these insights into smart traffic management systems can optimize urban infrastructure, reduce delays, fuel consumption, and emissions.

Ultimately, Connected Vehicle Data Analysis seeks to revolutionize transportation by developing smarter, safer, and more efficient systems. By utilizing IoT and machine learning, the project aims to improve the driving experience and contribute to societal benefits such as reducing environmental impact and enhancing public safety. This project demonstrates the potential of connected vehicles in shaping the future of mobility.

1.1 Motivation

The motivation behind this project is to address key challenges in modern transportation systems by leveraging connected vehicle data. As urban populations grow and traffic congestion increases, the need for smarter, more efficient transportation systems is critical. We aim to enhance vehicle communication with traffic systems, enabling seamless data exchange to support more informed decision-making. This improved communication can reduce congestion, streamline traffic flow, and enhance road safety by ensuring vehicles and traffic systems operate in sync, leading to smoother commutes and safer travel.

We focus on the rapid analysis of large-scale, real-time data, recognizing that efficient traffic management requires processing vast amounts of information quickly and accurately. By applying machine learning, we can analyze data in real-time, identify patterns, predict traffic conditions, and offer timely solutions to mitigate issues as they arise. This can reduce travel times, fuel consumption, and accidents, contributing to a more efficient and sustainable transportation system.

Additionally, protecting user privacy is a top priority while working with real-time data. As connected vehicles generate vast amounts of data, we ensure individuals' sensitive information is secure through encryption and anonymization protocols. Insights from vehicle performance and traffic patterns will help optimize fuel usage, reduce emissions, and promote eco-friendly driving.

By addressing these aspects, our goal is to create sustainable and intelligent transportation solutions that benefit both individuals and the environment. This project showcases the potential of connected vehicles to revolutionize urban mobility and create safer, more efficient, and environmentally friendly road networks.

1.1.1 Paper 1

The study *"Toward Achieving Fine-Grained Access Control of Data in Connected and Autonomous Vehicles"* by Jie Cui, published in the *IEEE Internet of Things Journal* (Volume 8, Issue 10, May 15, 2021; DOI: 10.1109/JIOT.2020.3041860), addresses the lack of secure access controls in connected and autonomous vehicles (CAVs). It proposes a fine-grained access control scheme (FGAC-inCAVs) to safeguard sensitive data like vehicle location and owner preferences. The system, featuring a trusted third party (TTP), perception components, and diverse applications, enhances security and privacy for in-vehicle data. [1].

1.1.2 Paper 2

The study *"Event Data Recorder and Transmitter for Vehicular Mishap Analysis Based on Sensor"* by Kumar.P Shiva and S. Muruganandam, presented at the *2023 ICCCI* (DOI: 10.1109/ICCCI56745.2023.10128628), explores using IoT and On-Board Diagnostics (OBD) in connected vehicles to analyze real-time driving data. It highlights leveraging this data to enhance safety, fuel efficiency, and cost-effective vehicular transportation. [3].

1.1.3 Paper 3

The study *"End-to-End Security Assessment Framework for Connected Vehicles"* by David Evans et al., presented at the *2019 22nd International Symposium on Wireless Personal Multimedia Communications (WPMC)* (DOI: 10.1109/WPMC48795.2019.9096062), introduces a framework for assessing security in connected vehicles. It emphasizes evaluating end-to-end vulnerabilities to address emerging threats in vehicle communication systems, ensuring robust protection for data and systems in connected automotive environments. [2].

1.1.4 Paper 4

The study *"Real-Time Life Analysis of the Electric Vehicles using IoT - Computer-Based Smart Systems"* by Bathala Neeraja et al., presented at the *2023 International Conference on Communication, Security and Artificial Intelligence (ICCSAI)* (DOI: 10.1109/ICCSAI10421637), examines IoT-based solutions for analyzing real-world EV usage data. It addresses challenges like infrastructure gaps and autonomy concerns, emphasizing IoT's role in improving the Smart Grid and ensuring accurate state of charge (SoC) predictions to prevent battery damage. The study evaluates techniques such as Node-Red and MQTT for effective implementation.[4]

1.2 Problem Statement

To Design and develop an integrated model for analyzing real-time data from connected vehicles, enabling efficient collection, transmission, and processing of insights.

1.3 Problem Analysis

The challenge in analyzing real-time data from connected vehicles lies in efficiently collecting, transmitting, and processing vast amounts of data generated by vehicle sensors. This includes information on speed, fuel efficiency, and environmental conditions. Existing systems may struggle with handling high-volume data streams, leading to delays or gaps in data transmission. Ensuring continuous, accurate data collection and high-speed transmission without interruptions is crucial to building a reliable, scalable system for real-time vehicle data analysis.

Once transmitted, processing the data in real-time presents another challenge. Analyzing large and complex datasets requires fast, accurate machine learning algorithms. Traditional systems may be too slow to provide timely insights for traffic management or vehicle diagnostics. Additionally, the system must be scalable to handle increasing data volumes while ensuring data privacy and security through encryption and anonymization. Balancing performance with privacy and security is essential in developing an effective real-time analysis model.

1.3.1 Design principle 1

Layered design pattern

One of the most well-known design patterns is perhaps the layered pattern. Without even understanding its name, many developers use it. The principle is to divide your code into "layers" where each layer has a certain duty and provides a higher layer with a service.

1.3.2 Scope and Constraints

Scope

- Real-time analysis of large-scale data from IoT-enabled vehicles to provide actionable insights.
- Enhancing traffic management, accident prevention, and predictive maintenance using machine learning models.
- Ensuring secure and efficient data transmission and storage to protect sensitive vehicle and user information.

Constraints

- Dependency on robust infrastructure for real-time data transmission, which may not be available in all areas.
- Ensuring privacy and security compliance while handling sensitive and personal data .
- Processing delays due to the massive volume of data generated by connected vehicles.
- Limited standardization of data formats and communication protocols across manufacturers.

1.4 Research Gap

One major challenge is the lack of standardized methods for integrating data from various vehicle platforms, sensors, and external sources like traffic systems and weather data. Different vehicles use varying sensor technologies and data formats, making it difficult to ensure interoperability across platforms. This lack of standardization leads to inefficiencies in data sharing, hindering the creation of a unified system for intelligent transportation, limiting the potential for real-time insights and decision-making.

Another gap lies in the scalability of real-time data processing. Current systems struggle to handle the massive volume of data generated by connected vehicles, often leading to delays or missed opportunities for timely action. The infrastructure needed to process this data, whether in cloud or edge computing systems, must be able to scale dynamically as data volumes grow, all while ensuring quick response times for applications like traffic management and predictive maintenance.

Furthermore, advanced predictive models are needed to accurately forecast road hazards and traffic conditions. Current models often lack the sophistication to handle complex, dynamic environments. Additionally, there is a lack of research on how connected vehicle data impacts transportation infrastructure, particularly in optimizing traffic flow, improving maintenance schedules, and influencing urban planning.

1.5 Objectives

- To generate vehicle data to simulate IoT sensor readings like speed, fuel consumption, tire pressure, etc.
- To apply machine learning algorithms like SVM and Random forest to predict traffic patterns and improve vehicle safety.
- To ensure secure data handling while optimizing energy consumption for sustainability.

Chapter 2

REQUIREMENT ANALYSIS

The requirement specification for the Connected Vehicle Data Analysis project outlines the technical specifications of the system, encompassing both hardware and software aspects. These specifications define the system's ability to process, analyze, and interpret real-time data collected from IoT-enabled vehicles. The functional requirements detail the expected performance metrics and the operational environment in which the system will function. This includes data collection from sensors, secure data transmission, and the application of machine learning models to derive actionable insights, ensuring the system meets the needs of connected vehicle ecosystems.

2.1 Functional Requirements

- The system shall be able to process the input from real-time vehicle data.
- System shall be able to optimize the solution for different datasets.
- The system shall provide an efficient method to analyze vehicle performance trends and driving behavior.
- The system shall identify the best predictive model for vehicle maintenance and performance insights.
- Perform result analysis.

2.2 Non Functional Requirements

- Compatibility: The application should work on any machine with the required configuration.
- Availability: The application should be available all time.
- Performance: The application must provide high performance.
- Efficiency: The application must have good final test accuracy after training the model.
- Reliability: The model should work for any computer-related component (software, hardware, or a network) that consistently performance according to its performs according to its specifications.

2.3 Software Requirements

Software requirements define the software resource requirements that need to installed on the system to provide the best functioning of an application.

- Jupyter notebook for Python - 3.12.2

  Jupyter Notebook is an open-source, interactive tool that combines coding, visualization, and documentation in a single environment. Supporting Python 3.12.2, it is ideal for data science, machine learning, and research projects. Users can write and execute live Python code alongside rich text, equations, and visualizations, which makes it easy to document workflows and share insights. The platform supports a variety of Python libraries, offering extensive functionality for tasks such as data manipulation, statistical modeling, and machine learning development. Jupyter's real-time code execution and debugging features streamline development, allowing users to test and iterate efficiently. Its browser-based interface makes it accessible and collaborative, enabling teams to share and modify notebooks with ease. Jupyter Notebook is particularly valued in academic and professional settings for creating reproducible research and detailed presentations of data-driven results.

- PowerBI for visualization

  Power BI by Microsoft is a powerful business intelligence tool designed for creating interactive dashboards and reports. It connects to a wide range of data sources, enabling users to transform raw data into meaningful insights. With its drag-and-drop interface, users can build dynamic visualizations, such as charts and graphs, without requiring advanced technical skills. Power BI supports real-time data analysis, providing updated insights for timely decision-making. It also facilitates collaboration by allowing teams to share and interact with reports through Power BI's cloud service. Its integration with other Microsoft tools, like Excel and Azure, enhances productivity and data accessibility. Whether for business analysis or research, Power BI simplifies complex datasets into visually engaging formats, helping users identify trends and patterns effectively.

2.4 Hardware Requirements

Hardware requirements refer to the physical components and specifications a computer must have to run specific software or perform certain tasks efficiently. These include components like CPU, RAM, storage, and graphics processing units (GPU) that ensure optimal performance and compatibility with the software.

- 8.00 GB

  A system with 8.00 GB of RAM is ideal for running modern software applications smoothly. RAM (Random Access Memory) is a critical component of a computer's hardware that temporarily stores data for active processes. With 8 GB, users can multitask efficiently, handling tasks such as coding, data analysis, and running applications like Jupyter Notebook or Power BI without significant slowdowns. This amount of memory is sufficient for working with moderate-sized datasets, performing machine learning experiments, and creating interactive visualizations. It also supports seamless operation of web browsers with multiple tabs, ensuring a smooth experience when researching or collaborating online. For most standard development and analytical tasks, 8 GB provides a balance of performance and affordability.

- AMD Ryzen 3

  The AMD Ryzen 3 processor is a reliable and budget-friendly CPU designed for everyday computing and entry-level performance tasks. Known for its efficiency, the Ryzen 3 features multiple cores and threads, allowing for smooth multitasking and the handling of lightweight development workloads. It is well-suited for running applications like Jupyter Notebook, handling Python scripts, and creating visualizations in Power BI. Its advanced architecture ensures good energy efficiency, reducing heat generation and power consumption. The processor is also equipped with integrated graphics, which can support visual tasks without requiring a separate graphics card. For students and professionals engaged in coding, data analysis, or business tasks, the AMD Ryzen 3 provides a cost-effective solution with dependable performance.

Chapter 3

SYSTEM DESIGN

3.1 Architectural Framework/System Design

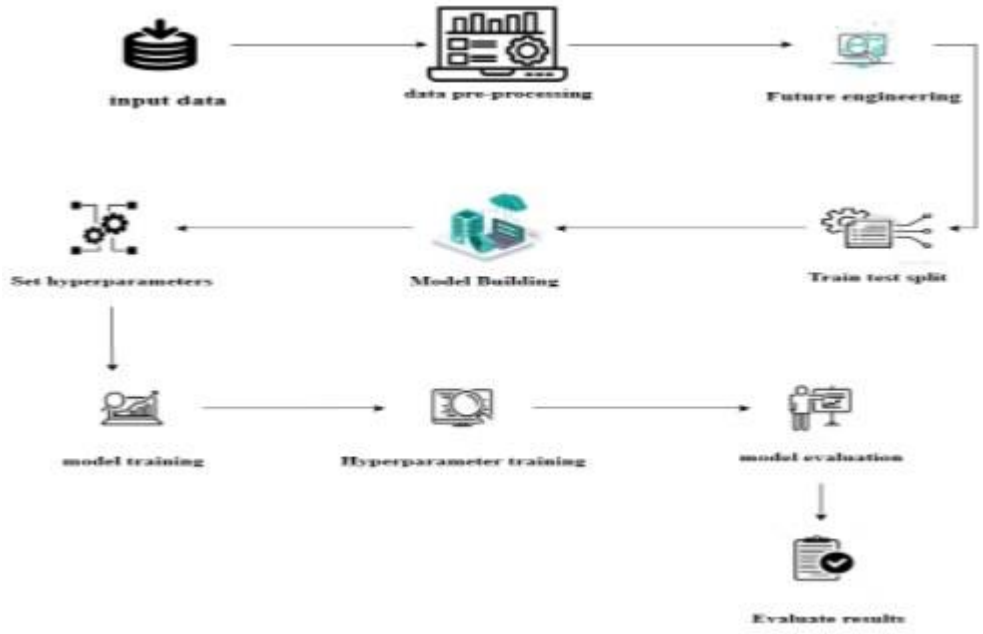

Figure 3.1: System design

This system architecture for Connected Vehicle Data Analysis illustrates the endto-end process of leveraging IoT-enabled vehicle data to detect and analyze critical events. It begins with Data Simulation, where real-world or synthetic data from connected vehicles is generated. This data encompasses various parameters, such as speed, fuel efficiency, GPS location, and sensor readings, serving as the foundation for subsequent analysis. The Data Preprocessing stage cleans and prepares this raw data by addressing noise, missing values, and inconsistencies, ensuring it is ready for accurate analysis and model training.

1. Input Data: The machine learning pipeline begins with gathering raw data from diverse sources, such as databases, APIs, sensors, or external datasets. This data can come in various forms, including structured data (e.g., tabular data, numerical values) and unstructured data (e.g., text, images, audio). The quality and variety of the input data play a crucial role in determining the

effectiveness of the subsequent model. Before it can be used for training, this data typically needs to undergo various processing steps to prepare it for analysis.

2. Data Pre-processing: Once the raw data is collected, it is essential to clean and transform it into a usable format for model training. Data pre-processing involves several key tasks: handling missing values (by imputation or removal), addressing outliers, and ensuring data consistency. The data may need to be normalized or standardized to ensure that numerical features have comparable scales. Categorical data is often encoded into numerical formats, such as one-hot encoding, to be used by machine learning algorithms. This stage ensures that the data is of high quality, reducing noise and improving model performance.

3. Feature Engineering: Feature engineering is a critical phase where domain expertise and creativity are applied to improve the data's predictive power. In this step, new features are generated from the existing data to capture additional insights that can help the model perform better. This could involve creating interaction terms between features, transforming variables (e.g., logarithmic transformations), or applying dimensionality reduction techniques such as Principal Component Analysis (PCA) to reduce the number of features while retaining essential information. Effective feature engineering can significantly improve the accuracy and efficiency of the model.

4. Train-Test Split: Once the data is pre-processed and features have been engineered, it is divided into two distinct subsets: the training set and the testing set. The training set is used to train the model, while the testing set is kept aside to evaluate its generalization performance. A common split ratio is 70-30 or 80-20, where 70

5. Model Building: In this phase, an appropriate machine learning model is selected and trained using the training data. The choice of model depends on the type of problem (e.g., classification, regression) and the nature of the data. Common models include decision trees, linear regression, support vector machines, random forests, and neural networks. During model building, the algorithm learns from the training data by adjusting its internal parameters to minimize errors and capture underlying patterns. The model is iteratively refined until it achieves an acceptable level of accuracy.

6. Set Hyperparameters: Hyperparameters are external parameters that govern the learning process of the machine learning model. These are not learned from the data but must be set manually before training. Examples include the learning rate, the number of layers in a neural network, or the maximum depth of a decision tree. Properly tuning these hyperparameters can significantly impact the performance of the model. Setting the right hyperparameters ensures that the model learns efficiently and doesn't underfit or overfit the data.

7. Model Training: The training phase involves fitting the model to the training data and adjusting its internal parameters to minimize the loss function. The model iteratively adjusts its parameters by learning from the data, making small updates to improve its accuracy. During training, the model learns patterns from the features and their relationships with the target variable. The model is trained until a stopping criterion is met, such as achieving an acceptable error rate or completing a predefined number of iterations.

8. Hyperparameter Training: After the model is trained, the next step involves fine-tuning the hyperparameters to improve the model's performance. Hyperparameter training typically involves searching for the optimal hyperparameter values using methods like grid search or random search. Grid search exhaustively tests all possible combinations of hyperparameters within a specified range, while random search tests a random subset of hyperparameters. This fine-tuning helps the model achieve better accuracy and robustness by finding the optimal settings for its learning process.

9. Model Evaluation: Once the model is trained and hyperparameters are tuned, it is evaluated using the testing dataset. The goal is to assess how well the model performs on data that it has not seen before. Common evaluation metrics include accuracy, precision, recall, F1 score, mean squared error (MSE), and area under the curve (AUC), depending on the type of problem (e.g., classification or regression). Model evaluation helps to determine whether the model has generalized well to unseen data or if it has overfitted to the training data.

10. Evaluate Results: The final step involves analyzing the evaluation results to determine if the model meets the desired performance criteria. If the model's performance is satisfactory, it may be deployed to production for making realtime predictions or used for further analysis. However, if the results are not as expected, adjustments are made to earlier stages, such as refining features, choosing a different model, or re-tuning the hyperparameters. Iterating through the pipeline ensures continuous improvement in the model's accuracy and relevance for realworld applications.

3.2 Data Set

The dataset used for this project consists of 100,001 data entries sourced from Kaggle.com, with seven class variables representing the key features of connected vehicle data. The dataset was divided into two subsets for model development and evaluation: 70% of the data was allocated for training the machine learning model, allowing it to learn patterns and relationships within the data, while the remaining 30% was reserved for testing, ensuring an unbiased assessment of the model's performance on unseen data. This balanced split ensures robust model training and reliable validation results.

3.3 Design Principles

Design principles play a crucial role in developing robust and scalable systems. These principles provide structured approaches to solving complex problems, ensuring efficient and maintainable designs. In the context of vehicle data analysis, leveraging these principles allows developers to handle large datasets and deliver insightful analytics effectively.

3.3.1 LAYERED DESIGN PATTERN

The layered design pattern is one of the most widely used design principles. It involves dividing the system into multiple layers, where each layer has a specific responsibility and interacts with adjacent layers to provide services. This modular approach ensures scalability, maintainability, and a clear separation of concerns.

For vehicle data analysis, the layered design can be applied as follows:

- Data Collection Layer: Responsible for collecting data from various sources such as IoT sensors, GPS systems, and vehicle diagnostics.
- Data Processing Layer: Prepares the raw data for analysis through steps like cleaning, normalization, and feature extraction.
- Analysis Layer: Implements machine learning models to identify patterns, make predictions, or detect anomalies in vehicle behavior or system performance.
- Visualization Layer: Provides user-friendly dashboards and visual representations of the analyzed data for effective decision-making.

This structured approach simplifies the integration of new functionalities, such as additional data sources or enhanced analysis techniques, ensuring the system remains adaptable and efficient over time.

Chapter 4

IMPLEMENTATION

Implementation, which is like a process that will turn plans and strategies into the actions so that we accomplish our objectives and goals. So to implement our application, we went through the mentioned processes to accomplish our goals. Below suggested methodology is the key to fulfill our objective tasks.

4.1 SVM Algorithm

The Support Vector Machine (SVM) algorithm is a powerful supervised machine learning method used for classification and regression tasks. It works by finding the optimal hyperplane that separates data points from different classes in a multidimensional feature space. This hyperplane acts as a decision boundary, ensuring that data points from one class are on one side and those from another class are on the opposite side. The algorithm maximizes the margin, which is the distance between the hyperplane and the closest data points from each class, known as support vectors. This maximization improves the model's ability to generalize to new data.

For datasets that are not linearly separable, SVM uses the kernel trick to transform the data into a higher-dimensional space where a linear hyperplane can effectively separate the classes. Common kernel functions include linear, polynomial, and radial basis function (RBF) kernels, which allow SVM to handle complex relationships in the data.

SVM is particularly well-suited for high-dimensional datasets and performs well even when the number of features exceeds the number of samples. It is effective at preventing overfitting through regularization parameters like C, which balances training accuracy and model simplicity. However, SVM can be computationally intensive for large datasets and requires careful tuning of parameters and kernel selection.

SVM is widely applied in various domains, including image recognition (e.g., face detection), text classification (e.g., spam filtering), and bioinformatics (e.g., gene classification). Its strength lies in creating clear decision boundaries, making it an excellent choice for tasks requiring high precision and generalization.

4.1.1 Key Parameters Used in the SVM Model

- Kernel: Specifies the function to transform data into a higher-dimensional space for better separation.
- C: Balances training accuracy and model simplicity to avoid overfitting.
- Gamma: Controls the influence of a single data point on the decision boundary.
- Degree: Defines the complexity of the polynomial kernel for non-linear data.
- Max Iterations: Limits the number of iterations for model training convergence.
- Random State: Ensures reproducibility of the training process by setting a fixed seed.

Figure 4.1: Algorithmic flowchart of SVM

- Start: Initiates the SVM workflow to classify or predict based on input data.
- Data Normalization: Scales input features to ensure consistency and improve model accuracy.
- Input Data: Represents the raw dataset with features and labels for training.
- Kernel Representation: Maps data to a higher-dimensional space for better class separability.
- Optimization of Kernel Data: Uses mathematical techniques to find the optimal hyperplane in the transformed space.
- SVM Training Process: Learns the decision boundary by maximizing the margin between classes.
- Performance Criteria: Evaluates the model's accuracy, precision, recall, and other metrics.
- No: If criteria aren't met, the model is refined or retrained.
- Yes: Proceeds to the prediction simulation phase.

- Prediction Simulation: Applies the trained SVM model to new data for classification.
- Statistical Indications: Provides key metrics to interpret the model's effectiveness and reliability.
- End: Concludes the process with predictions and performance insights.

Since Support Vector Machine (SVM) relies on finding the optimal hyperplane for classification, inappropriate feature scaling or irrelevant features can negatively impact model performance. To improve accuracy and stability, we propose a feature selection and normalization phase that emphasizes significant features and ensures consistent scaling, enhancing the robustness of the SVM model.

4.2 Random forest Algorithm

The Random Forest algorithm is a powerful ensemble learning method used for classification and regression tasks, and it is highly effective when applied to the analysis of connected vehicle data. This algorithm builds multiple decision trees, each trained on a random subset of the data, and then aggregates the results to improve predictive accuracy and reduce overfitting. In the context of connected vehicle data, which involves various real-time parameters such as speed, fuel efficiency, engine performance, and environmental conditions, Random Forest is particularly effective due to its ability to handle high-dimensional and noisy data. The trees in the forest are constructed independently, considering different subsets of features, and each tree makes a prediction based on the patterns it has learned.

the Random Forest algorithm is utilized to extract meaningful insights from the large and complex datasets generated by connected vehicles. It starts by performing feature selection, identifying the most important variables that contribute to accurate predictions. During training, the model learns from the patterns and relationships within the data, improving its ability to classify vehicle states or predict events like maintenance needs or fuel consumption. Once trained, the model tests the data and aggregates predictions from all the individual trees to generate a final result. This combination of multiple decision trees ensures that Random Forest provides more reliable and stable predictions compared to single decision trees. Additionally, the algorithm's ability to manage missing data and handle imbalanced datasets further strengthens its suitability for real-time vehicle data analysis, ultimately providing actionable insights to enhance vehicle performance, safety, and traffic management.

4.2.1 Key Parameters Used in the Random Forest Model

- n_estimators: The number of trees in the forest, controlling the model's complexity and accuracy.

- max_depth: Limits the depth of each tree to prevent overfitting.
- random_state: Sets a seed for reproducibility in model training.
- x_train, y_train: Training feature matrix (x_train) and target vector (y_train) for model learning.
- x_test, y_test: Test feature matrix (x_test) and target vector (y_test) for model evaluation.

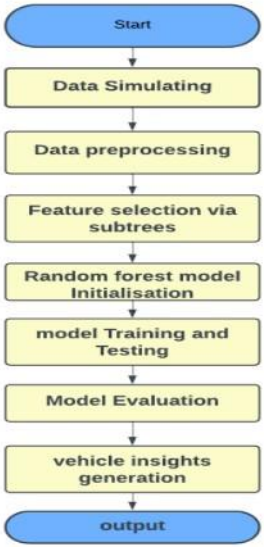

Figure 4.2: Algorithmic flowchart of Random Forest

- Data Simulation: Generate synthetic vehicle data with key parameters.
- Data Preprocessing: Clean and normalize the data for analysis.
- Feature Selection via Subtrees: Identify important features using decision tree splits.
- Random Forest Model Initialization: Set parameters like tree count and depth.
- Model Training and Testing: Train the model on training data and test it on unseen data.
- Model Evaluation: Evaluate the model using performance metrics like accuracy.
- Vehicle Insights Generation: Generate predictions for vehicle performance or maintenance.
- Output: Provide actionable insights for vehicle management and traffic optimization.

Since Random Forest relies on an ensemble of decision trees, random initialization can lead to overfitting on irrelevant features. To improve performance, we propose a feature selection phase that prioritizes important features, enhancing model stability and accuracy.

Chapter 5

RESULTS AND DISCUSSIONS

This study, we compared the performance of Support Vector Machines (SVM) and Random Forest classifiers in predicting energy consumption classes. The Random Forest model achieved a perfect accuracy score of 1.00, while the SVM classifier scored 0.9877. Both models showed strong performance with a macro average and weighted average of 1.00 for Random Forest and 0.99 for SVM, indicating their ability to handle the classification task effectively. The confusion matrices reveal that both models classified the energy consumption classes (Low, Medium, High) with minimal misclassifications, demonstrating robust prediction accuracy. Additionally, the data distribution plots indicate a balanced dataset across energy consumption classes. The Principal Component Analysis (PCA) plots of the decision boundaries for SVM and Random Forest clearly show separations between the classes, suggesting that both models are capable of discerning complex patterns in the feature space. While Random Forest outperforms SVM slightly, both classifiers are highly effective for predicting energy consumption levels. These results highlight the reliability and potential of these machine learning algorithms for real-world applications in energy prediction, with further improvements achievable through hyperparameter tuning and exploration of additional models.

5.1 TEST CASE 1

METRIC	SVM	RANDOM FOREST
ACCURACY	0.9877	1.00
MACRO AVG	0.99	1.00
WEIGHTED AVG	0.99	1.00

Figure 5.1: Model Performance Comparison Metrics

Random Forest:

- Accuracy: 0.9877, slightly lower than Random Forest, but still a very high value indicating good performance.
- Macro Average: 0.99, indicating that SVM also performs well across all classes, with a slight drop compared to Random Forest.

- Weighted Average: 0.99, also indicating good overall performance, though slightly less balanced than Random Forest.
- Performance: The SVM classifier performs well, with a very high accuracy and strong averages, but it shows slightly lower performance compared to Random Forest, particularly in terms of the weighted average.

Support Vector Machine (SVM):

- Accuracy: 0.9877, slightly lower than Random Forest, but still a very high value indicating good performance.
- Macro Average: 0.99, indicating that SVM also performs well across all classes, with a slight drop compared to Random Forest.
- Weighted Average: 0.99, also indicating good overall performance, though slightly less balanced than Random Forest.
- Performance: The SVM classifier performs well, with a very high accuracy and strong averages, but it shows slightly lower performance compared to Random Forest, particularly in terms of the weighted average.

Results of Test Case 1

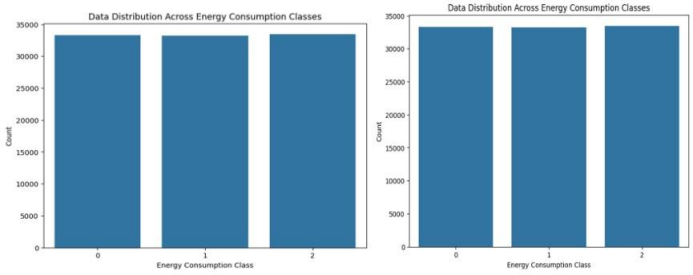

Figure 5.2: Data Distribution Across Energy Consumption Classes

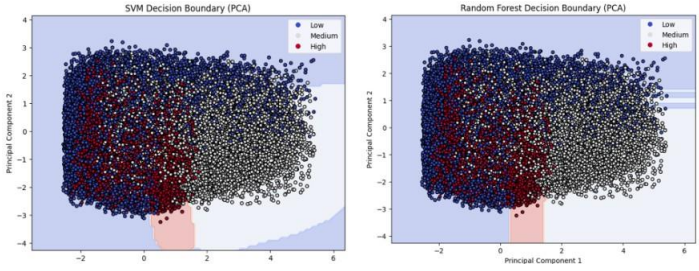

Figure 5.3: Decision Boundary

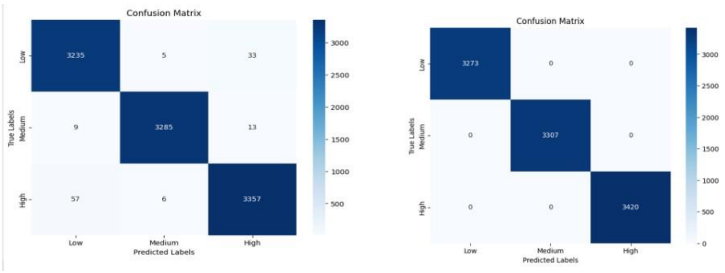

Figure 5.4: Confusion Matrix

5.2 TEST CASE 2

METRIC	SVM	RANDOM FOREST
ACCURACY	0.9864	0.9993
MACRO AVERAGE	0.99	1.00
WEIGHTED AVERAGE	0.99	1.00

Figure 5.5: Model Performance Comparison Metrics

Random Forest:

- Accuracy: 0.9993, indicating nearly perfect classification with minimal errors.
- Macro Average: 1.00, suggesting excellent performance across all classes equally.
- Weighted Average: 1.00, showing perfect balance between precision and recall across all class sizes.
- Performance: Random Forest performs exceptionally well with high accuracy and perfect averages across all metrics.

Support Vector Machine (SVM):

- Accuracy: 0.9864, slightly lower than Random Forest, but still very high performance.
- Macro Average: 0.99, indicating good performance across all classes with a minor drop compared to Random Forest.
- Weighted Average: 0.99, showing good overall performance, but slightly less balanced than Random Forest.
- Performance: SVM performs well but is slightly outperformed by Random Forest in terms of accuracy and balance.

Results of Test Case 1

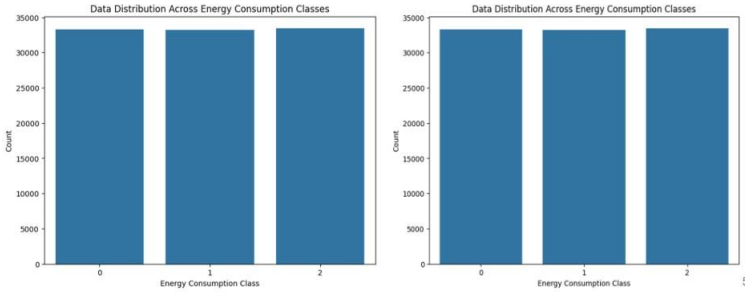

Figure 5.6: Data Distribution Across Energy Consumption Classes

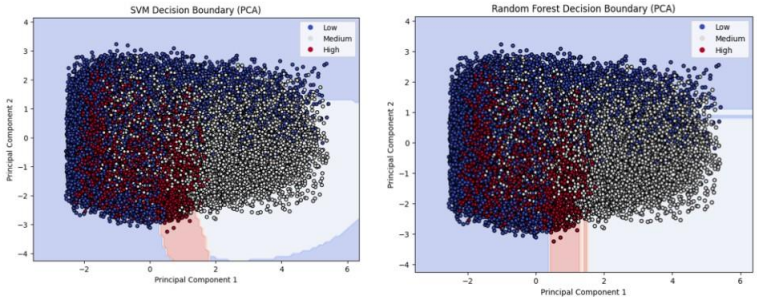

Figure 5.7: Decision Boundary

5.3 TEST CASE 3

METRIC	SVM	RANDOM FOREST
ACCURACY	0.99	1.00
MACRO AVERAGE	0.99	1.00
WEIGHTED AVERAGE	0.99	1.00

Figure 5.9: Model Performance Comparison Metrics

Random Forest:

- Accuracy: 1.00, demonstrating perfect classification performance with no errors across all classes.
- Macro Average: 1.00 and Weighted Average: 1.00, indicating flawless balance and performance across all classes, with no class imbalance.

Support Vector Machine (SVM):

- Accuracy: 0.99, slightly lower than Random Forest but still indicating strong performance with minimal errors.
- Macro Average: 0.99 and Weighted Average: 0.99, showing good performance across all classes, though slightly less balanced compared to Random Forest.

Results of Test Case 1

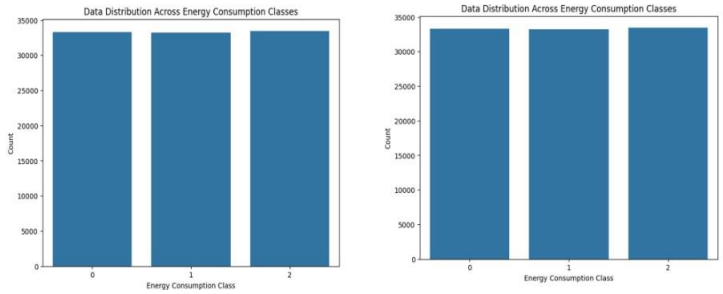

Figure 5.10: Data Distribution Across Energy Consumption Classes

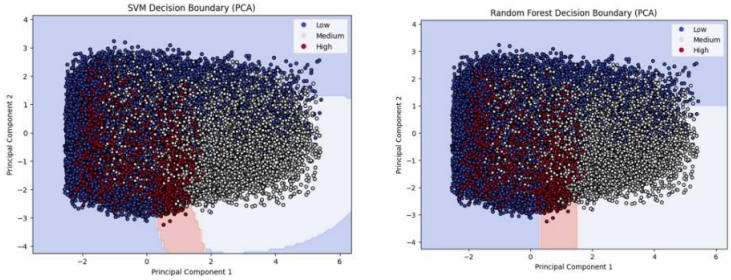

Figure 5.11: Decision Boundary

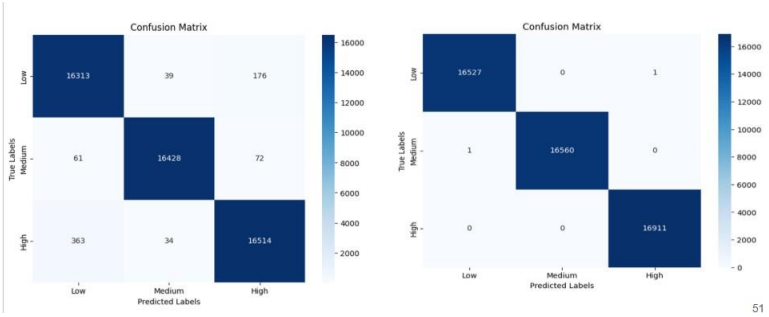

Figure 5.12: Confusion Matrix

5.4 TEST CASE 4

METRIC	SVM	RANDOM FOREST
ACCURACY	0.9735	0.9998
MACRO AVERAGE	0.97	1.00
WEIGHTED AVERAGE	0.97	1.00

Figure 5.13: Model Performance Comparison Metrics

Random Forest:

- Accuracy: 0.9998, indicating near-perfect classification with almost no errors.
- Macro Average: 1.00, showing equal performance across all classes without any imbalance.

- Weighted Average: 1.00, demonstrating perfect balance between precision and recall, regardless of class frequency.
- Performance: Random Forest performs excellently with near-perfect scores across all metrics, showcasing its high effectiveness in handling the classification task.

Support Vector Machine (SVM):

- Accuracy: 0.9735, slightly lower than Random Forest, indicating strong performance but with a few more misclassifications.
- Macro Average: 0.97, showing good performance across all classes with a minor drop from Random Forest.
- Weighted Average: 0.97, reflecting strong overall performance, though slightly less balanced than Random Forest.
- Performance: SVM performs very well but slightly lags behind Random Forest in terms of accuracy and balance.

Results of Test Case 1

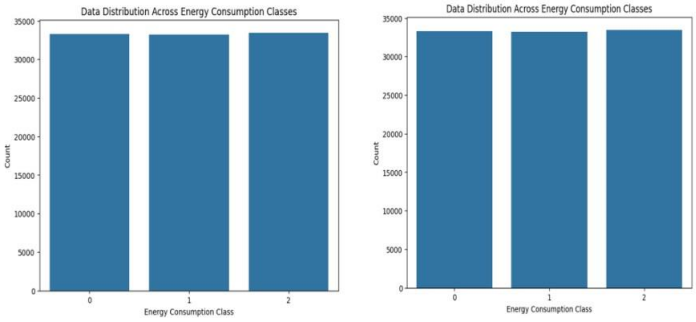

Figure 5.14: Data Distribution Across Energy Consumption Classes

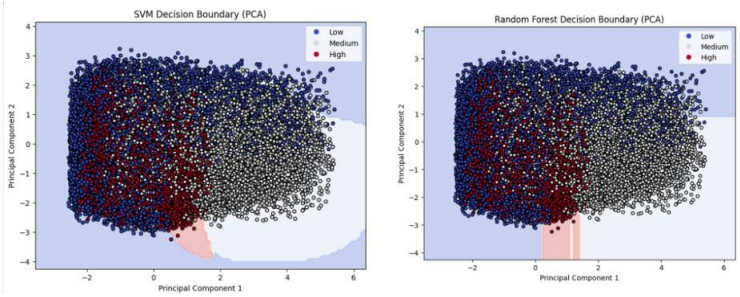

Figure 5.15: Decision Boundary

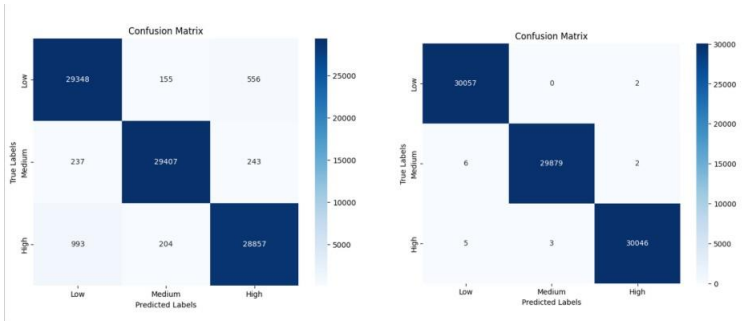

Figure 5.16: Confusion Matrix

Chapter 6

RESULTS AND DISCUSSIONS

Across all four test cases, the Random Forest model consistently outperforms the Support Vector Machine (SVM) in terms of classification accuracy. SVM's accuracy varies from 0.9735 to 0.99, showing incremental improvements with each test case. However, Random Forest achieves near-perfect accuracy, consistently ranging from 0.9993 to 1.0 across all conditions. This indicates that Random Forest is highly reliable and performs exceptionally well in classifying the given data, achieving near flawless results, while SVM, although effective, has a slightly lower range of accuracy.

When examining the decision boundaries of both models, Random Forest demonstrates superior class separation. The decision boundaries for SVM, although reasonable, show some overlap between classes, especially in the earlier test cases. This overlap can lead to higher instances of misclassification, as the SVM struggles with classifying instances in regions where the data points are less distinct. In contrast, Random Forest provides clearer and sharper decision boundaries, making it more effective at differentiating between classes, particularly when the data is complex or has subtle distinctions.

The confusion matrices further emphasize the differences in performance between the two models. SVM, while performing well overall, shows noticeable misclassifications across the test cases, particularly in the earlier conditions where class boundaries are less clear. Random Forest, on the other hand, exhibits minimal or no misclassifications, reinforcing its ability to classify data accurately. This highlights Random Forest's robustness and reliability, as it consistently achieves high accuracy without misclassifications, positioning it as the superior classification model when compared to SVM across all four test cases.

COMPARISION ANALYSIS

SVM struggled with non-linear relationships and had lower accuracy in vehicle data due to its single decision boundary. In contrast, Random Forest effectively handled complex features, providing higher accuracy and better performance.

- comparison highlights the performance differences between SVM and Random Forest models in vehicle data analysis. SVM, relying on a single decision boundary, struggled with non-linear relationships and had lower accuracy. It was less effective in capturing the complexities of

dynamic real-world data. In contrast, Random Forest handled complex features better, providing higher accuracy.

- Random Forest outperformed SVM by using ensemble decision-making, making it more adaptable to real-time scenarios. Its ability to handle complex, non-linear relationships helped it process vehicle data effectively. By aggregating decisions from multiple trees, Random Forest achieved higher accuracy and reduced misclassifications. This makes it a better choice for detailed vehicle data analysis and real-time decision-making.

ASPECT	SVM	RANDOM FOREST
Application to Project	Less effective for real-time vehicle data due to reliance on a single decision boundary.	Excelled with higher accuracy using ensemble decision-making.
Handling Vehicle Data	Struggled with non-linear relationships in features like speed and energy consumption.	Adapted well to complex, non-linear data relationships.
Accuracy	Lower accuracy due to misclassification outside the boundary.	Consistently higher accuracy from aggregated tree decisions.

Table 6.1: Comparison table

Chapter 7

CONCLUSION

In this project, we developed a model to analyze real-time data from connected vehicles, focusing on key features such as speed, distance, and energy consumption. The primary goal was to evaluate the performance of machine learning algorithms in processing this dynamic and complex data. Our evaluation showed that the Random Forest algorithm consistently outperformed SVM in terms of accuracy. This was due to Random Forest's ensemble approach, which aggregates multiple decision trees to make more robust and reliable predictions. On the other hand, SVM, which relies on a single decision boundary, struggled with the non-linear relationships in the data, resulting in more frequent misclassifications. The results highlighted that Random Forest's ability to handle complex, non-linear patterns made it more effective for analyzing real-time vehicle data. Therefore, Random Forest proved to be a better choice for ensuring accurate and reliable predictions in dynamic, real timescenarios. This demonstrates the importance of using ensemble methods in real-time systems for better performance and reliability. Additionally, Random Forest's flexibility in adapting to various data patterns further enhances its applicability in connected vehicle data analysis.

Chapter 8

Future Work

For future work, we plan to integrate advanced machine learning algorithms such as XGBoost and LightGBM, which are known for their efficiency in handling large datasets and improving model accuracy. These gradient boosting algorithms are particularly effective at reducing bias and variance, and we aim to compare their performance with Random Forest to identify the most suitable algorithm for predicting vehicle performance in real-time.

Additionally, we will focus on incorporating real-time data streaming using platforms like Apache Kafka or Spark Streaming. These technologies will enable the model to process large volumes of connected vehicle data efficiently, allowing for real-time predictions and quicker decision-making. This capability will be crucial for applications requiring immediate analysis, such as detecting maintenance needs or adjusting vehicle performance during trips.

Furthermore, we plan to expand the dataset by adding features such as traffic conditions, driver behavior, and environmental factors (e.g., weather or road conditions). Including these variables will provide more comprehensive insights into vehicle performance, enabling more accurate predictions and improving both safety and efficiency. By integrating these advancements, the system will be more capable of delivering real-time, actionable insights for connected vehicles.

Bibliography

[1] J. Cui, X. Chen, J. Zhang, Q. Zhang and H. Zhong, *"Toward Achieving Fine-Grained Access Control of Data in Connected and Autonomous Vehicles*" .in IEEE Internet of Things Journal, vol. 8, no. 10, pp. 7925-7937, 15 May15, 2021, doi: 10.1109/JIOT.2020.3041860.

[2] K. P. Shiva and S. Murugaanandam., *"Event Data Recorder And Transmitter For Vehicular Mishap Analysis Based On Sensor,* .2023 International Conference on Computer Communication and Informatics (ICCCI), Coimbatore, India, 2023, pp. 1-6, doi: 10.1109/ICCCI56745.2023.10128628.

[3] D. Evans, D. Calvo, A. Arroyo, A. Manilla and D. Gómez, *"End-to-end security assessment framework for connected vehicles,"* .2019 22nd International Symposium on Wireless Personal Multimedia Communications (WPMC), Lisbon, Portugal, 2019, pp. 1-6, doi: 10.1109/WPMC48795.2019.9096062.

[4] B. Neeraja, H. Ralte, K. Jain, S. R. Surlekar, D. A. Arani and R. C. Mabborang, *"Real-Time Life Analysis of the Electric Vehicles using IoT - Computer-Based Smart Systems,"*. 2023 International Conference on Communication, Security and Artificial Intelligence (ICCSAI), Greater Noida, India

[5] 5. W. Lu, Z. Yi, B. Liang, Y. Rui and B. Ran, *"Learning Car-Following Behaviors for a Connected Automated Vehicle System: An Improved Sequence-to-Sequence Deep Learning Model",*. in IEEE Access, vol. 11, pp. 28076-28089, 2023, doi: 10.1109/ACCESS.2023.3243620.